Automated Payroll Processing & Payslip Delivery Engine

Region: ap-south-2

Prepared by

D Mohamed Fadhil

Swagata Sinha

M.t Nishanth

Submission Package

1. Revision History

Version	Date	Prepared By	Remarks
1.0	Submission Draft	D Mohamed Fadhil / Swagata Sinha / M.t Nishanth	final technical design document

2. Document Structure

- Executive Summary
- Project Overview
- Business Problem
- Functional Requirements
- Non-Functional Requirements
- Assumptions and Constraints
- High-Level Architecture
- Component Design
- Data Model Design
- API Specification
- Payroll Calculation Logic
- Idempotency Design
- PDF Generation Design
- Email Delivery Design
- Monitoring and Logging
- Security Design
- Failure Handling and Recovery
- Testing Strategy
- Team Split and Integration Agreement
- Deployment and Configuration Guide
- Dry Run Walkthrough
- Cost Considerations
- Future Enhancements
- Conclusion

3. Executive Summary

The Automated Payroll Processing & Payslip Delivery Engine is a serverless payroll automation solution designed on AWS. It removes manual spreadsheet-based payroll processing and replaces it with an auditable, scalable, event-driven workflow.

The system accepts a payroll run request, identifies active employees, distributes one processing message per employee through Amazon SQS, calculates salary and deductions in parallel Lambda workers, generates PDF payslips, stores the documents in Amazon S3, and emails secure download links through Amazon SES.

The solution is intentionally built to demonstrate real cloud engineering concepts such as event-driven fan-out, immutable audit records, idempotent processing, secure document delivery, least-privilege IAM, and observability with CloudWatch.

4. Project Overview

The project simulates an enterprise payroll flow. HR initiates a monthly payroll run from a simple static dashboard hosted in S3. The dashboard calls API Gateway, which invokes a payroll trigger Lambda. That Lambda loads active employees from DynamoDB and publishes one SQS message per employee.

A separate worker Lambda processes each employee record independently. This separation allows the solution to scale as the employee count grows, while also making the architecture simpler to maintain and easier to explain in an interview or internship review.

The final output of each payroll run is an immutable payroll record, a PDF payslip, a pre-signed download URL, and an SES email sent to the employee.

5. Business Problem

Payroll is often one of the most sensitive and time-critical business processes in an organization. Manual spreadsheet-based payroll introduces a number of risks: formula errors, duplicated entries, delayed payslip distribution, poor traceability, and difficult audit handling.

As the number of employees increases, a manual approach becomes slower and less reliable. A cloud-native solution is preferred because it can process records in parallel, preserve a complete run history, and reduce the chance of operational mistakes.

This project addresses those issues by using managed AWS services instead of custom servers or monolithic applications.

6. Functional Requirements

- Maintain salary component records per employee in DynamoDB.
- Maintain deduction records per employee in DynamoDB.
- Maintain immutable payroll run records in DynamoDB.
- Trigger payroll manually through API Gateway.
- Use SQS to fan out one payroll message per employee.
- Process employee payroll records in parallel with Lambda workers.
- Calculate gross pay, deductions, and net pay using agreed business rules.
- Generate PDF payslips using ReportLab in a Lambda Layer.
- Store payslip PDFs in Amazon S3.
- Generate 7-day pre-signed URLs for document access.
- Send payslip emails through Amazon SES.
- Provide a minimal static HR dashboard for payroll initiation, history, links, and CSV export.

7. Non-Functional Requirements

- Scalability: support parallel processing through SQS and Lambda.
- Reliability: avoid duplicate payroll processing using idempotency.
- Auditability: keep payroll records immutable after finalization.
- Security: use least-privilege IAM, private S3 storage, and pre-signed URLs.
- Observability: capture operational logs and errors in CloudWatch.
- Maintainability: keep components loosely coupled and contract-driven.

8. Assumptions and Constraints

The original project requirement clearly defined the salary, deduction, and delivery workflow, but several implementation details were intentionally left open. To keep all team members aligned, the following assumptions are fixed for the final design.

- AWS Region: ap-south-2.
- Employee status values: ACTIVE, INACTIVE, TERMINATED, ON_LEAVE.
- Only ACTIVE employees are processed during a payroll run.
- Professional tax slabs are simplified for the project and use three tiers.
- TDS is fixed at 10% of gross salary.
- The payroll run identifier format is RUNYYYYMM.
- The payroll_runs table uses run_id as partition key and employee_id as sort key.
- The dashboard is intentionally minimal and single-page only.

9. High-Level Architecture

The architecture is shown in the attached diagram and follows a clean serverless event-driven pattern. The user interacts with a static web dashboard hosted in S3. The dashboard calls API Gateway, which invokes the payroll trigger Lambda. The trigger Lambda reads active employees and publishes one SQS message per employee.

Amazon SQS acts as the fan-out buffer and decouples payroll initiation from worker execution. Worker Lambdas then process employee payroll in parallel, reading source data from DynamoDB, applying idempotency checks, calculating salary, generating PDF payslips, storing them in S3, and sending emails through SES.

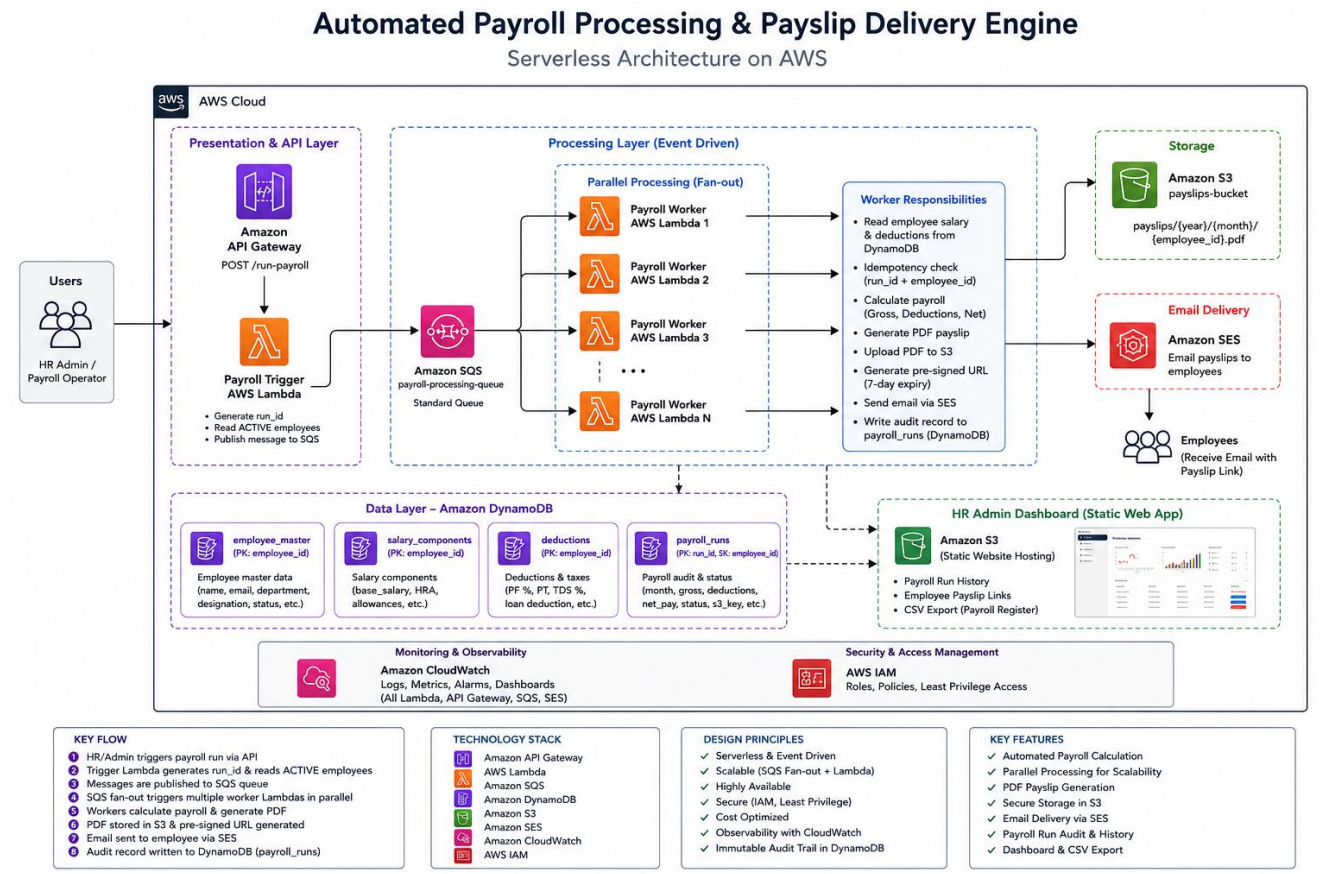

Figure 1: Automated Payroll Processing & Payslip Delivery Engine Architecture

10. Component Design

10.1 Amazon API Gateway

- Receives the payroll trigger request from the dashboard.
- Exposes the POST /run-payroll endpoint.
- Invokes the payroll trigger Lambda synchronously.
- Keeps the user interface decoupled from business logic.

10.2 Payroll Trigger Lambda

- Generates the payroll run identifier using the agreed RUNYYYYMM format.
- Reads the employee master table to find all ACTIVE employees.
- Creates one SQS message per employee.
- Publishes messages to the payroll-processing-queue.
- Returns an acknowledgement that payroll processing has started.

10.3 Amazon SQS

- Acts as the fan-out layer for the system.
- Buffers employee work items and smooths out load spikes.
- Enables independent processing of each employee payroll task.

- Supports retries and decoupling between trigger and worker components.

10.4 Payroll Worker Lambda

- Consumes one message per employee from SQS.
- Reads salary and deduction data from DynamoDB.
- Performs idempotency checks before any processing.
- Calculates gross salary, deductions, and net salary.
- Writes the payroll result to payroll_runs.
- Generates a payslip PDF and uploads it to S3.
- Creates a pre-signed URL and sends email through SES.

10.5 Amazon DynamoDB

- Stores the employee master data.
- Stores salary component and deduction data.
- Stores payroll output records in an immutable design.
- Acts as the system of record for the project.

10.6 Amazon S3

- Stores generated payslip PDFs securely.
- Hosts the static dashboard.
- Uses a folder pattern based on year, month, and employee_id.
- Works with pre-signed URLs for controlled document access.

10.7 Amazon SES

- Sends payslip notification emails to employees.
- Delivers a secure 7-day download link instead of public file access.
- Separates document generation from document delivery.

10.8 Amazon CloudWatch

- Collects logs from Lambda functions.
- Captures execution success, failures, and exception traces.
- Supports debugging, audit support, and operational visibility.

10.9 AWS IAM

- Controls permissions for Lambda, SQS, S3, SES, and DynamoDB access.
- Applies least-privilege security to each service role.
- Prevents accidental overexposure of payroll data.

11. Data Model Design

The data model is intentionally compact and clearly separated by responsibility. The source data tables are keyed by employee_id. The payroll_runs table is keyed by run_id and employee_id so that all records for a payroll cycle can be queried together.

employee_master

Primary key: employee_id.

Key attributes: employee_name, email, department, designation, employment_status, joining_date.

Example record:


```
{"employee_id":"EMP001","employee_name":"John
Doe","email":"john.doe@company.com","department":"Engineering","designation":"Cloud
Engineer","employment_status":"ACTIVE","joining_date":"2024-01-15"}
```


salary_components

Primary key: employee_id.

Key attributes: base_salary, hra, transport_allowance, special_allowance.

Example record:


```
{"employee_id":"EMP001","base_salary":50000,"hra":20000,"transport_allowance":5000,"special_allowanc
e":3000}
```


deductions

Primary key: employee_id.

Key attributes: pf_percentage, professional_tax, tds_percentage, loan_deduction.

Example record:


```
{"employee_id":"EMP001","pf_percentage":12,"professional_tax":300,"tds_percentage":10,"loan_deductio
n":2000}
```


payroll_runs

Primary key: run_id (PK), employee_id (SK).

Key attributes: month, gross, deductions_total, net_pay, status, payslip_s3_key.

Example record:


```
{"run_id":"RUN202606","employee_id":"EMP001","month":"2026-
06","gross":78000,"deductions_total":16100,"net_pay":61900,"status":"COMPLETED","payslip_s3_key":"pa
yslips/2026/06/EMP001.pdf"}
```


12. API Specification

The front-end dashboard triggers payroll through a single HTTP API. The API is intentionally minimal because its purpose is to start a payroll run, not to implement business logic.

Method	Resource	Description
POST	/run-payroll	Starts the payroll processing flow and returns run_id

Request body example is optional for this project because the trigger Lambda can derive the required processing context from the active employee list.

13. Payroll Calculation Logic

The payroll calculation logic is deterministic and derived from the stored employee salary and deduction data.

- Gross Salary = Base Salary + HRA + Transport Allowance + Special Allowance
- PF = 12% of Base Salary
- Professional Tax: salary <= 25000 => 0, 25001 to 50000 => 200, above 50000 => 300
- TDS = 10% of Gross Salary
- Loan deduction is read directly from the deductions table
- Net Salary = Gross - PF - PT - TDS - Loan Deduction

For the golden test employee, the expected outcome is Gross = 78000, PF = 6000, PT = 300, TDS = 7800, Loan = 2000, Total Deductions = 16100 and Net Salary = 61900.

14. Idempotency Design

Idempotency is one of the most important parts of the project because the payroll workflow uses SQS and Lambda, both of which may retry operations. Without idempotency, duplicate messages could produce duplicate payroll runs, duplicate PDFs, or duplicate email notifications.

The shared idempotency key format is run_id#employee_id. Before processing a worker message, the Lambda checks whether a payroll_runs record already exists for that run and employee. If the record exists, the message is skipped. If it does not exist, processing continues.

This design ensures that every employee is processed exactly once per payroll run from the perspective of the application, even when the underlying platform may deliver messages more than once.

15. PDF Generation Design

The payslip must be produced as a well-formatted PDF document using ReportLab packaged in a Lambda Layer. The worker Lambda generates the PDF after payroll calculation completes and before the email is sent.

- Include company name, employee name, employee ID and payroll month.
- Show earnings breakdown with base salary, HRA, transport allowance and special allowance.
- Show deductions breakdown with PF, professional tax, TDS and loan deduction.
- Display gross salary and net salary clearly.
- Generate the file at /payslips/{year}/{month}/{employee_id}.pdf.

The generated S3 object key is written back to payroll_runs so the dashboard and future audits can reference the stored document location.

16. Email Delivery Design

Amazon SES is used to send the payslip notification email to each employee. The email contains a secure pre-signed S3 link that expires after seven days. This prevents publicly exposing the document and keeps access controlled without requiring a separate download service.

The email body is intentionally simple: greeting, download link, and expiry notice. This keeps the user experience clear and avoids overcomplicating the workflow.

17. Monitoring and Logging

CloudWatch is used to capture logs from the trigger Lambda and the worker Lambda. Logging should include run start, employee processing, calculation success, PDF creation, S3 upload, email send success, and exception traces.

- Payroll started
- Employee processing started
- Idempotency check result
- Calculation completed
- PDF generated
- PDF uploaded to S3
- SES email sent
- Failure or exception details

Operational logging is important because the payroll pipeline is asynchronous, and CloudWatch is the primary place to trace a run during debugging or audit review.

18. Security Design

Payroll data is highly sensitive and must remain private. The solution uses IAM least privilege, private S3 storage, and pre-signed URLs for controlled document sharing.

- S3 buckets must not be public.
- Lambda roles must only have the permissions they actually need.
- DynamoDB tables should be accessed only by the application roles.
- SES should use verified sender identities.
- Pre-signed URLs should have a short, fixed expiration window.

19. Failure Handling and Recovery

A payroll system must distinguish between calculation failure, document generation failure, and email delivery failure.

- If payroll calculation fails, the payroll_runs status is set to FAILED.
- If PDF generation fails, the payroll_runs status is set to FAILED.
- If email delivery fails after payroll succeeds, payroll_runs remains COMPLETED and the error is logged.
- Failed SQS messages can be retried according to Lambda and SQS behavior.

This ensures that financial calculation results are not lost even if the notification step fails.

20. Testing Strategy

- Unit tests for salary calculation, deduction calculation and net salary computation.
- Integration tests for API Gateway to Lambda to SQS to worker flow.
- End-to-end tests covering payroll initiation, PDF creation, S3 upload and SES delivery.
- Idempotency tests to verify duplicate message handling.
- Data validation tests for missing or malformed employee records.

The golden test employee should be used in every environment so that all team members validate against the same expected result.

21. Team Split and Integration Agreement

The work is divided into three balanced parts so that each team can build independently while still integrating cleanly with the others.

Team	Ownership	Deliverables
Team 1	DynamoDB, S3, sample data, data contracts	Tables, bucket, schema documentation, golden employee
Team 2	Static website, API Gateway, trigger Lambda, SQS	Dashboard, trigger flow, queue, API contract
Team 3	Worker Lambda, payroll engine, PDF generation, SES, CloudWatch	Payroll processing, PDFs, email, logs, final output

All teams must follow the same shared contracts for table names, key design, run_id format, SQS message format, status values and environment variables.

22. Deployment and Configuration Guide

1. Create the DynamoDB tables and load sample data.
2. Create the S3 bucket for payslips and dashboard hosting.
3. Create the SQS queue and configure the trigger Lambda event source mapping.
4. Create the API Gateway endpoint and connect it to the trigger Lambda.
5. Deploy the worker Lambda with the ReportLab layer attached.
6. Configure SES sender identity and email permissions.
7. Set environment variables consistently across both Lambda functions.
8. Deploy the static dashboard to S3 and configure its API endpoint.
9. Run the golden employee dry run and verify the payroll outcome.

23. Dry Run Walkthrough

The dry run below demonstrates how the system behaves during a real payroll run.

10. HR clicks Run Payroll on the static dashboard.

11. The dashboard calls API Gateway.
12. API Gateway invokes the payroll trigger Lambda.
13. The trigger Lambda generates run_id RUN202606.
14. The trigger Lambda loads all ACTIVE employees.
15. The trigger Lambda publishes one SQS message per employee.
16. SQS delivers each employee message to a worker Lambda.
17. The worker Lambda checks idempotency using run_id#employee_id.
18. The worker Lambda calculates payroll, generates PDF and uploads it to S3.
19. The worker Lambda creates a pre-signed URL and sends SES email.
20. The worker Lambda inserts the payroll_runs record.
21. CloudWatch stores operational logs for the run.

24. Cost Considerations

This architecture is cost-efficient because it uses managed services and only consumes resources when payroll is actually processed.

- Lambda charges are usage-based and fit well for periodic payroll runs.
- SQS is inexpensive for lightweight payroll messaging.
- DynamoDB can be used in on-demand mode for simple scaling.
- S3 is low cost for storing PDF documents.
- SES is cost-effective for transactional email delivery.

25. Future Enhancements

- Add authentication and authorization for the dashboard.
- Add payroll approval workflow before finalization.
- Add richer tax rules and region-specific tax logic.
- Add employee self-service access to payslip history.
- Add analytics for payroll trends and monthly comparisons.
- Integrate with enterprise HRMS or ERP systems.

26. Conclusion

The Automated Payroll Processing & Payslip Delivery Engine demonstrates a complete serverless AWS solution for a critical business workflow. It shows how to combine API Gateway, Lambda, SQS, DynamoDB, S3, SES, IAM and CloudWatch into a secure, scalable and auditable payroll platform.

The project is intentionally designed to be easy to present, easy to test and realistic enough to demonstrate the core skills expected in cloud engineering and solutions architecture work.

Appendix A. Sample Payroll Record

{"run_id":"RUN202606","employee_id":"EMP001","month":"2026-06","gross":78000,"deductions_total":16100,"net_pay":61900,"status":"COMPLETED","payslip_s3_key":"payslips/2026/06/EMP001.pdf"}

Appendix B. Shared Integration Contracts

- Table names: employee_master, salary_components, deductions, payroll_runs.
- Bucket name: payroll-payslips.
- Region: ap-south-2.
- SQS message format: { run_id, employee_id }.
- Payroll status values: PROCESSING, COMPLETED, FAILED.
- Employee status values: ACTIVE, INACTIVE, TERMINATED, ON_LEAVE.
- Environment variables: EMPLOYEE_TABLE, SALARY_TABLE, DEDUCTIONS_TABLE, PAYROLL_RUNS_TABLE, PAYROLL_QUEUE_URL, PAYSLIP_BUCKET, SES_SENDER_EMAIL.

Appendix C. Requirements Traceability Matrix

This matrix maps the major requirements from the project brief to the AWS components and implementation artifacts used in the solution.

Requirement	Implemented By	Artifact	Status
Payroll trigger	API Gateway + Trigger Lambda	POST /run-payroll	Implemented
Employee fan-out	Amazon SQS	payroll-processing-queue	Implemented
Parallel processing	Worker Lambda	SQS event source mapping	Implemented
Payroll calculations	Worker Lambda	Calculation module	Implemented
Payslip PDF generation	ReportLab in Lambda Layer	PDF builder	Implemented
Payslip storage	Amazon S3	payslips/{year}/{month}/{employee_id}.pdf	Implemented
Email delivery	Amazon SES	Transactional email template	Implemented
Audit trail	Amazon DynamoDB	payroll_runs table	Implemented

Appendix D. DynamoDB Access Patterns

The data model is designed around the access patterns required by the payroll workflow. The system reads employee records, salary components, deductions, and payroll results using employee_id and run_id as the primary lookup values.

Access Pattern	Table	Why It Is Needed
Find active employees	employee_master	Trigger Lambda needs the employee list
Load salary by employee	salary_components	Worker Lambda calculates gross salary
Load deductions by employee	deductions	Worker Lambda calculates tax and loan deductions
Query payroll history by run	payroll_runs	Dashboard and audit report payroll results
Prevent duplicate payroll processing	payroll_runs	Idempotency check before insert

This design keeps the tables simple, readable, and optimized for the exact workflow the project implements.

Appendix E. IAM Role and Permission Matrix

Each AWS service role should be granted only the permissions required for its own responsibilities. This reduces accidental exposure of payroll data and keeps the deployment aligned with least-privilege principles.

Role / Component	Required Permissions	Purpose
Trigger Lambda role	DynamoDB read, SQS send, CloudWatch logs	Read active employees and push queue messages
Worker Lambda role	DynamoDB read/write, S3 put/get, SES send, CloudWatch logs	Process payroll and deliver payslips
Static website hosting	S3 read for website assets	Host dashboard files
API Gateway integration	Invoke Lambda	Start payroll execution

The IAM design should be implemented with separate roles for trigger and worker functions rather than a shared broad role. This reduces blast radius if a single component is misconfigured.

Appendix F. API Request and Response Examples

The API is intentionally minimal. The dashboard only needs to start the payroll workflow and receive acknowledgement that processing has begun.

Request	Example
Method	POST
Endpoint	/run-payroll
Purpose	Start payroll execution
Response	{"run_id":"RUN202606","message":"Payroll processing started"}

The front-end can use the run_id in the response to display confirmation and later fetch payroll history records from DynamoDB-backed reporting logic.

Appendix G. CloudWatch Log Samples

CloudWatch logs are critical for tracing asynchronous execution. The following examples show the type of messages that should appear in the Lambda logs.

- Payroll trigger started for RUN202606.
- Found 3 ACTIVE employees.
- SQS message created for EMP001.
- Worker received message for RUN202606#EMP001.
- Payroll calculation completed successfully.
- PDF uploaded to s3://payroll-payslips/payslips/2026/06/EMP001.pdf.

- SES email sent successfully to john.doe@company.com.

When a failure occurs, the log should include the employee_id, run_id, and the step at which the failure occurred so that troubleshooting can be done quickly.

Appendix H. Failure Scenarios and Recovery Strategy

Failure Scenario	Expected Behavior	Recovery Action
Missing salary record	Mark payroll as FAILED	Fix data and rerun payroll
PDF generation exception	Mark payroll as FAILED	Inspect layer/package and retry
SES send failure	Keep payroll COMPLETED	Log email failure and resend manually if needed
Duplicate SQS delivery	Skip due to idempotency	No action required
Temporary DynamoDB throttle	Retry according to Lambda/SQS behavior	Monitor throughput and reprocess

The failure strategy separates payroll computation errors from notification errors so that a completed payroll calculation is not lost due to a downstream messaging issue.

Appendix I. Test Cases

Test Case	Input	Expected Output	Status
Golden employee calculation	EMP001 data	Net pay = 61900	Pass criteria
Duplicate message	Same run_id#employee_id twice	Second attempt skipped	Pass criteria
Missing salary record	No salary row	FAILED status	Pass criteria
Email delivery success	SES available	Email sent + URL delivered	Pass criteria
Email delivery failure	SES unavailable	Payroll remains COMPLETED	Pass criteria

Each team member should execute these test cases before integration sign-off. The golden employee should be used as the primary verification record.

Appendix J. Deployment Runbook

22. Create the AWS resources in the final approved order.
23. Load the employee master data and validate active records.
24. Load salary component and deduction records.
25. Create the S3 bucket and upload the dashboard files.
26. Deploy API Gateway and connect the trigger Lambda.
27. Attach SQS event source mapping to the worker Lambda.

28. Deploy the ReportLab Lambda Layer and validate PDF creation.
29. Verify SES sender identity and test email sending.
30. Run the golden employee payroll test.
31. Validate CloudWatch logs and payroll_runs entries.

The deployment runbook is important because it gives the team a repeatable sequence for provisioning and verifying the solution during final demo preparation.

Appendix K. Cost Considerations

Service	Cost Characteristic	Comment
Lambda	Usage-based	Cost is low for monthly payroll runs
SQS	Request-based	Very inexpensive for one message per employee
DynamoDB	On-demand or provisioned	On-demand is simplest for a project
S3	Storage and requests	Payslip PDFs are small and inexpensive to store
SES	Email send usage	Suitable for transactional payslip delivery

Because the architecture is serverless, the project avoids the fixed cost of always-on compute resources and remains efficient for internship-scale demonstrations.

Appendix L. Detailed Dry Run Example

This example shows one full path for EMP001 during RUN202606.

32. Trigger Lambda generates RUN202606.
33. Trigger Lambda identifies EMP001 as ACTIVE.
34. Trigger Lambda sends an SQS message with {run_id, employee_id}.
35. Worker Lambda receives the message and performs the idempotency check.
36. Worker Lambda reads salary_components and deductions for EMP001.
37. Worker Lambda calculates gross = 78000 and net pay = 61900.
38. Worker Lambda writes the payroll_runs record with status COMPLETED.
39. Worker Lambda generates the PDF and stores it in S3.
40. Worker Lambda creates a 7-day pre-signed URL.
41. Worker Lambda sends the SES email to john.doe@company.com.

If the same message is delivered again, the idempotency check will detect the existing payroll_runs entry and skip processing safely.

Appendix M. Acceptance Criteria

- A payroll run can be started from the dashboard.
- One SQS message is created for each active employee.
- Each worker processes one employee independently.
- A PDF payslip is created and stored in S3.
- An SES email is delivered with a pre-signed URL.
- Payroll run records remain immutable after finalization.
- The dashboard can display payroll history and export CSV data.
- CloudWatch logs show successful execution and any errors.

The project should be considered complete only when all acceptance criteria are demonstrated in the final end-to-end dry run.

Appendix N. Glossary

Term	Meaning
Run ID	Unique identifier for one payroll execution cycle
Idempotency	Property that prevents duplicate processing of the same request
Fan-out	Distribution of one workload into many parallel tasks
Pre-signed URL	Temporary link that grants controlled access to an S3 object
Immutable record	A record that is not updated after creation

Appendix O. Data Dictionary

Table	Attribute	Type / Meaning	Notes
employee_master	employee_id	String	Partition key
employee_master	employment_status	String	ACTIVE / INACTIVE / TERMINATED / ON_LEAVE
salary_components	base_salary	Number	Monthly base pay
salary_components	special_allowance	Number	Additional earnings component
deductions	pf_percentage	Number	Used to compute PF from base salary
deductions	loan_deduction	Number	Fixed deduction amount per employee
payroll_runs	gross	Number	Calculated gross salary
payroll_runs	payslip_s3_key	String	S3 object key of the PDF

This dictionary allows each team to validate field names and expected value types before integration. It is especially useful when one member is generating sample data while another is implementing Lambda logic.

Appendix P. Expanded IAM and Permission Notes

The solution uses separate permissions per component so that each Lambda can only perform the actions it must execute. The trigger Lambda does not need write access to S3, and the worker Lambda does not need permission to create API Gateway resources.

Component	Read	Write	Reason
Trigger Lambda	employee_master	SQS	Creates work items only
Worker Lambda	salary_components, deductions	payroll_runs, S3, SES	Performs payroll and delivery
Dashboard	payroll_runs	None	Read-only payroll visibility
Static website	HTML/CSS/JS assets	None	Hosting only

This separation makes the system easier to reason about and reduces accidental data modification paths.

Appendix Q. Operational Checklist

- Verify all DynamoDB tables exist before testing.
- Confirm the employee master contains at least one ACTIVE employee.
- Validate that the SQS queue is connected to the worker Lambda.
- Check that the PDF generation layer is attached to the worker Lambda.
- Confirm SES sender identity is verified.
- Ensure the S3 bucket is private and has the correct folder structure.
- Review CloudWatch logs after each test payroll run.
- Verify the dashboard uses the correct API endpoint.

This checklist should be completed before the final demo so that avoidable environment issues do not affect the submission.

Appendix R. Sequence of Events

Step	Event
1	User clicks Run Payroll in the dashboard
2	API Gateway invokes the trigger Lambda
3	Trigger Lambda generates the monthly run_id
4	Trigger Lambda queries active employees
5	Trigger Lambda sends one SQS message per employee
6	Worker Lambda consumes the SQS message
7	Worker Lambda checks idempotency and loads payroll data
8	Worker Lambda calculates salary and writes payroll_runs
9	Worker Lambda generates PDF, uploads to S3, and sends SES email
10	CloudWatch captures logs for audit and troubleshooting

This event sequence is the operational backbone of the solution and should be used as the basis for the demo explanation.

Appendix S. Sign-Off Checklist

- Architecture diagram reviewed and accepted.
- Table schemas agreed by all team members.
- SQS message contract frozen.
- run_id format frozen.
- Idempotency key design frozen.
- Region set to ap-south-2.
- Golden test employee verified.
- Team split accepted by all members.

Once this checklist is complete, the team can proceed to implementation without ambiguity in the shared contracts.

Appendix T. Example Payroll Register CSV Format

The payroll register CSV is intended for HR download and audit review. It should be generated from payroll_runs data and contain one row per employee per payroll run.

run_id	month	employee_id	gross	deductions_total	net_pay	status
RUN202606	2026-06	EMP001	78000	16100	61900	COMPLETED

The CSV should be generated only from finalized payroll_runs entries so that the export stays consistent with the immutable audit record.

Appendix U. Payroll Run Review Checklist

Review Item	Verification
Active employee count	Match expected number of employees queued
Payroll totals	Validate gross and net salary against sample calculation
PDF output	Confirm S3 object exists and opens correctly
Email delivery	Confirm SES message and pre-signed link
Audit trail	Confirm payroll_runs entry is immutable and complete

This review checklist should be completed after every test run and before the final demo video is recorded.